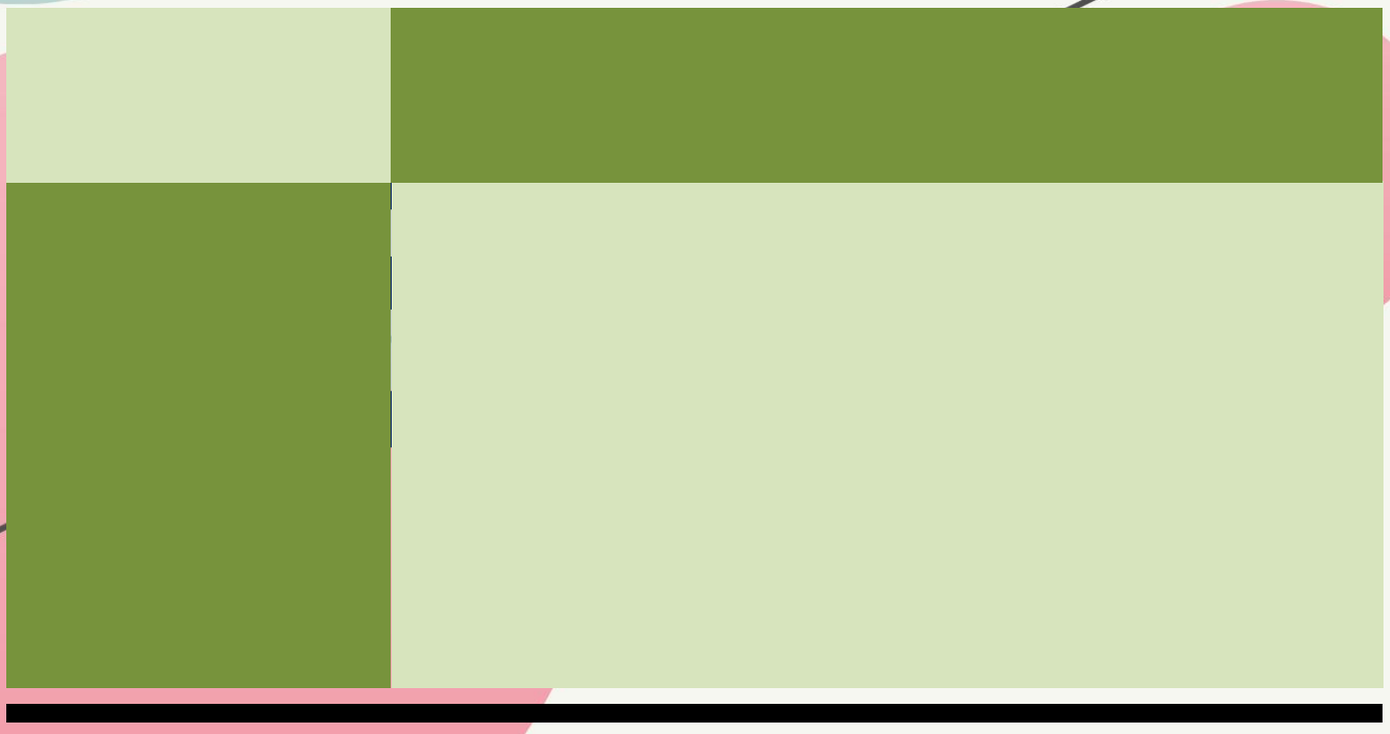


File Management

LEARNING OUTCOMES

By the end of this module, students will be able to:

1. Identify common file allocation methods, File System Structure: Directory Structures, File Allocation Method and analyze the advantages and limitations of different directory and file allocation strategies
2. Analyze how disk scheduling, RAID, and file system structures contribute to data reliability and system performance
3. Analyze the strengths and limitations of various file systems

Table of Contents

5.1 File System Structure: Directory Structures, File Allocation Methods 4

Directory Structures 4

File Allocation Methods 5

Advantages and Disadvantages 6

Real-World Examples 7

5.2 Storage Management: Disk Scheduling Algorithms, RAID, File System 8

Disk Scheduling Algorithms 8

RAID (Redundant Array of Independent Disks) 10

Types of RAID Levels 11

Benefits of Using RAID in Modern Data Storage Solutions 12

Benefits of Using RAID in Modern Data Storage Solutions 13

Limitations and Risks of RAID 15

RAID in Modern Operating Systems 16

Hardware-Based RAID in Enterprise Environments 18

File System Implementation 19

Key Components 19

Common File Systems 21

File System Operations 23

Storage Management System 24

5.3 Analysis of File Systems in Modern Operating Systems 28

5.4 SUMMARY 32

5.1 File System Structure: Directory Structures, File Allocation Methods

A file system is a core component of an operating system that organizes, manages, and stores data on storage devices. It provides a structured approach for storing, retrieving, and managing files and directories.

Directory Structures

A directory structure arranges files and folders hierarchically to simplify data management and access. It serves as a map for locating files on storage media.

Types of Directory Structures:

- **Single-Level Directory:** All files stored in a single directory. Simple but difficult to scale with many files.
- **Two-Level Directory:** Each user has a separate directory, reducing filename conflicts but offering limited complexity.
- **Tree-Structured Directory:** A hierarchical model supporting files and subdirectories. Easier to organize but complex in deep structures.
- **Acyclic Graph Directory:** Extends the tree structure by enabling shared files using links, though it risks dangling pointers.
- **General Graph Directory:** Allows multiple links and shared directories with cycles, but requires complex management to avoid loops.

Functions of Directory Structures:

- Organization: Hierarchically manages files and folders.
- Navigation: Provides logical paths for file access.
- Security: Facilitates access control and permissions.
- Efficiency: Enhances file retrieval speed.

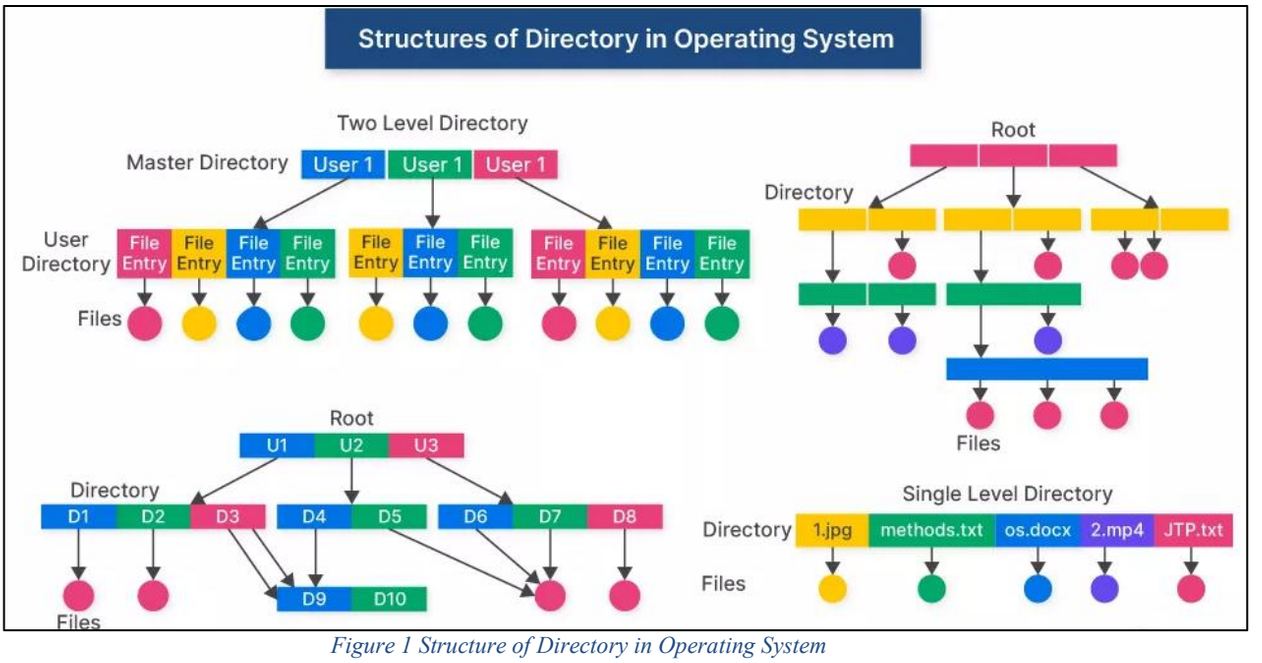


Figure 1 Structure of Directory in Operating System

File Allocation Methods

File allocation methods determine how data is stored on disks and how the OS keeps track of file locations.

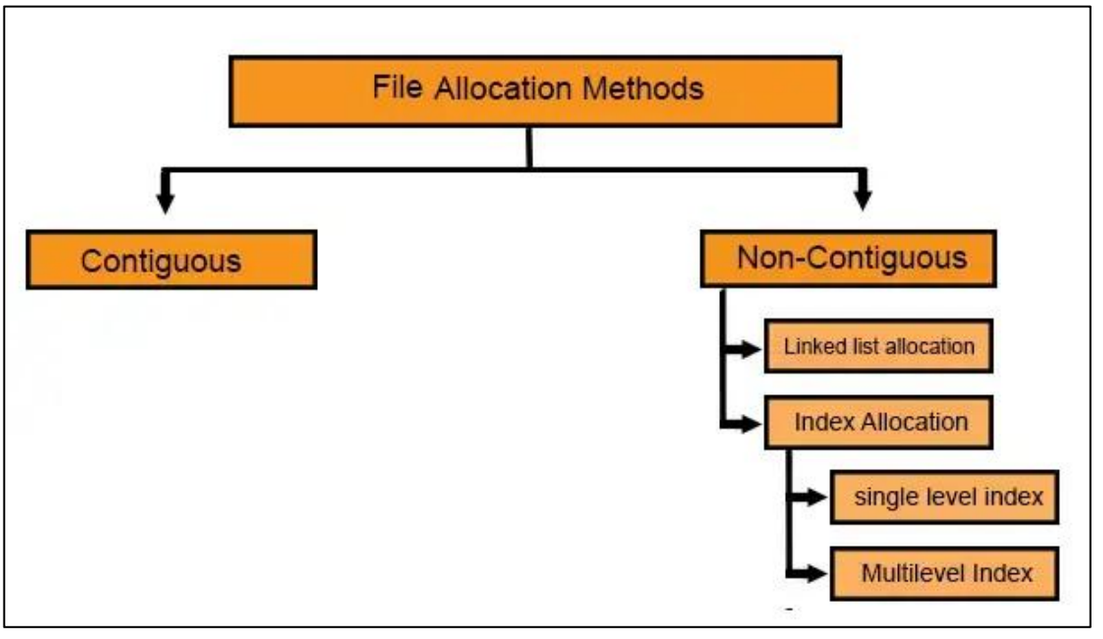


Figure 2 File Allocation Methods

Types of File Allocation Methods:

1. Contiguous Allocation: Files are stored in continuous memory blocks. Fast access but prone to fragmentation.
2. Linked Allocation: Files are stored as linked blocks scattered across the disk, making file expansion easy but access slower.
3. Indexed Allocation: A separate index block holds pointers to all file blocks, enabling direct access but limited by the index size.

Comparison:

Feature	Contiguous	Linked	Indexed
Storage Efficiency	Poor	Good	Good
Access Speed	Fast	Slow	Fast
File Expansion	Difficult	Easy	Easy
Implementation	Simple	Moderate	Complex

Figure 3 Comparison between Contiguous, linked and Indexed

Advantages and Disadvantages

Advantages:

- Tree and graph structures simplify organization.
- Indexed and linked allocation support large datasets.
- Graph structures facilitate data sharing.

Disadvantages:

- Graph structures add complexity.
- Linked allocation can slow large file access.
- Contiguous allocation suffers from fragmentation.

Real-World Examples

1. NTFS (Windows): Indexed allocation with B-tree structure.
2. FAT32 (Older Windows): Linked allocation.
3. EXT4 (Linux): Indexed allocation with extent-based storage.
4. HFS+ (MacOS): Tree-structured with indexed allocation.

The choice of directory structures and file allocation methods impacts file management efficiency, speed, and reliability. Contiguous allocation offers speed, indexed allocation provides flexibility, and linked allocation allows dynamic file growth. Tree and graph structures improve file organization and access control, making them essential in modern operating systems.



Here are some useful YouTube videos File Management in OS.

1. File Management Example. Watch here
2. File System Management in OS. Watch here
3. File Management in OS Watch here
4. Importance of File Management system. Watch here

5.2 Storage Management: Disk Scheduling Algorithms, RAID, File System

Storage Management is essential in an operating system (OS) for managing data storage and retrieval on physical drives efficiently. It optimizes disk performance, ensures data redundancy, and maintains proper file organization for smooth system operation.

Disk Scheduling Algorithms

Disk scheduling algorithms determine how disk I/O (Input/Output) requests are handled to improve access time and performance.

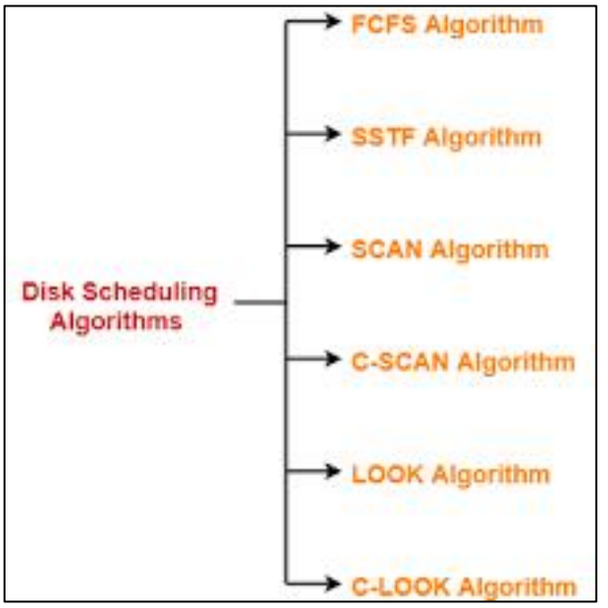


Figure 4 Disk Scheduling Algorithm

Why Disk Scheduling Matters:

- Disk access is slower compared to CPU processing speeds.
- Multiple I/O requests can create delays if not managed properly.
- Efficient scheduling reduces seek time and improves throughput.

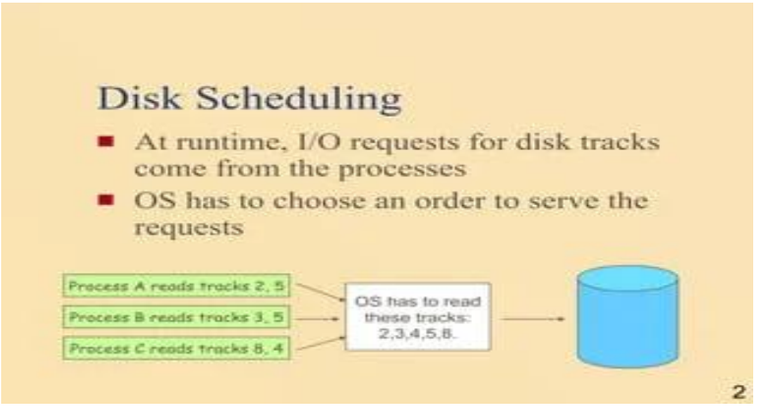


Figure 5 Disk Management System

Common Disk Scheduling Algorithms:

1. FCFS (First-Come, First-Served): Processes requests in order of arrival.
   - Pros: Simple to implement.
   - Cons: Long wait times and excessive head movement.

2. SSTF (Shortest Seek Time First): Processes the nearest request next.
   - Pros: Reduced seek time.
   - Cons: Risk of starvation for distant requests.

3. SCAN (Elevator Algorithm): Moves the disk head in one direction, serving all requests before reversing.
   - Pros: Prevents starvation and offers balanced performance.
   - Cons: Higher latency for edge requests.

4. C-SCAN (Circular SCAN): Moves in one direction and resets after reaching the end.
   - Pros: Uniform wait times.
   - Cons: More back-and-forth movement.

5. LOOK and C-LOOK: Variants of SCAN and C-SCAN that stop at the last request before reversing.
    - Pros: Reduced movement compared to standard SCAN and C-SCAN

Comparison Table:

Algorithm	Seek Time	Starvation	Fairness	Complexity
FCFS	High	No	Fair	Low
SSTF	Moderate	Yes	Poor	Moderate
SCAN	Moderate	No	Good	Moderate
C-SCAN	Moderate	No	Good	Moderate
LOOK	Low	No	Good	Moderate

Figure 6 Comparison Table for FCFS, SSTF, SCAN, C-SAN and LOOK

RAID (Redundant Array of Independent Disks)

RAID, which stands for Redundant Array of Independent Disks, is a data storage virtualization technology that combines multiple physical disk drives into a single logical unit to improve performance, reliability, and fault tolerance. It is widely used in modern computing environments to protect data against hardware failures and enhance the overall efficiency of data storage systems.

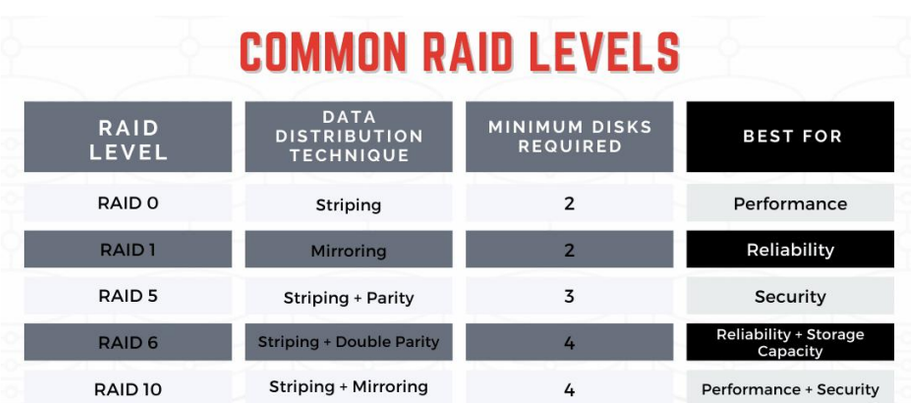


Figure 7 Common RAID levels

RAID was developed to address the limitations of single-disk storage systems, such as data loss due to disk failure and limited performance. By distributing data across multiple disks, RAID provides a balance between data redundancy (protection against failure) and increased storage performance.

The primary objectives of RAID are:

- Redundancy: Ensuring data availability in case of hardware failure.
- Performance: Increasing data transfer speeds through parallel data access.
- Capacity: Combining multiple disks into a larger, single logical volume.

Types of RAID Levels

RAID is categorized into multiple levels, each with distinct strategies for distributing data and redundancy.

RAID 0: Striping (No Redundancy)

- How it works: Data is split evenly across multiple disks (striped).
- Advantages: Increased read and write speed since data is accessed in parallel.
- Disadvantages: No fault tolerance—if one disk fails, all data is lost.
- Use Case: High-speed data access where redundancy is not critical (e.g., temporary cache storage).

RAID 1: Mirroring

- How it works: Data is duplicated across two disks. Both disks hold identical copies of data.
- Advantages: Excellent fault tolerance since data is fully mirrored.
- Disadvantages: Reduces available storage to half as data is duplicated.
- Use Case: Critical data storage where redundancy is essential, such as financial records.

RAID 5: Striping with Parity

- How it works: Data is striped across multiple disks with parity information stored on one disk. Parity allows data to be reconstructed if a disk fails.
- Advantages: Balanced between redundancy and performance, requires a minimum of three disks.
- Disadvantages: Parity calculation can reduce write performance.
- Use Case: Common for file servers and databases requiring both redundancy and performance.

RAID 6: Striping with Double Parity

- How it works: Similar to RAID 5 but stores two sets of parity information, allowing the system to recover from two simultaneous disk failures.
- Advantages: Increased fault tolerance compared to RAID 5.
- Disadvantages: Reduced write performance due to extra parity calculations.
- Use Case: Enterprise storage environments requiring high fault tolerance.

RAID 10 (RAID 1+0): Mirroring and Striping (Hybrid)

- How it works: Combines RAID 1 and RAID 0 by mirroring data and then striping across multiple disks.
- Advantages: High performance and fault tolerance.
- Disadvantages: Requires a minimum of four disks and results in reduced usable storage.
- Use Case: Mission-critical applications requiring both performance and redundancy.

Comparison of RAID Levels:

RAID Level	Performance	Fault Tolerance	Storage Efficiency	Use Case
RAID 0	High	None	100%	Speed-focused tasks
RAID 1	Moderate	High	50%	Data redundancy
RAID 5	Good	Moderate	67%-83%	General storage
RAID 6	Good	High	50%-67%	Critical data storage
RAID 10	Very High	High	50%	High-speed critical apps

Figure 8 Comparison of RAID Levels

Benefits of Using RAID in Modern Data Storage Solutions

RAID (Redundant Array of Independent Disks) is a critical technology in modern data storage that provides enhanced performance, fault tolerance, and data protection through the combination of multiple physical disks into a single logical unit. It is widely implemented in personal computing, data centers, and enterprise environments due to its ability to manage data efficiently and safeguard against data loss. The primary benefits of RAID include fault tolerance, improved performance, scalability, and data protection, each of which plays a crucial role in data management and system reliability

1. Fault Tolerance

Fault tolerance refers to the system's ability to continue functioning despite the failure of one or more components. In the context of RAID, fault tolerance is achieved through data redundancy, where copies of data or parity information are stored across multiple disks.

RAID levels such as RAID 1 (Mirroring) and RAID 5 (Striping with Parity) offer strong fault tolerance by either duplicating data or distributing parity information. If a disk fails, the system can recover the lost data using the redundant copies or parity stored on the other disks. This prevents data loss and ensures continuous system operation, which is particularly vital in environments where data availability is critical, such as financial institutions and healthcare systems.

By minimizing downtime and preventing complete system failures, RAID fault tolerance enhances business continuity and operational stability.

Benefits of Using RAID in Modern Data Storage Solutions

RAID (Redundant Array of Independent Disks) is a critical technology in modern data storage that provides enhanced performance, fault tolerance, and data protection through the combination of multiple physical disks into a single logical unit. It is widely implemented in personal computing, data centers, and enterprise environments due to its ability to manage data efficiently and safeguard against data loss. The primary benefits of RAID include fault tolerance, improved performance, scalability, and data protection, each of which plays a crucial role in data management and system reliability.

1. Fault Tolerance

Fault tolerance refers to the system's ability to continue functioning despite the failure of one or more components. In the context of RAID, fault tolerance is achieved through data redundancy, where copies of data or parity information are stored across multiple disks.

RAID levels such as RAID 1 (Mirroring) and RAID 5 (Striping with Parity) offer strong fault tolerance by either duplicating data or distributing parity information. If a disk fails, the system can recover the lost data using the redundant copies or parity stored on the other disks. This prevents data loss and ensures continuous system operation, which is particularly vital in environments where data availability is critical, such as financial institutions and healthcare systems.

By minimizing downtime and preventing complete system failures, RAID fault tolerance enhances business continuity and operational stability.

2. Improved Performance

Performance improvement is another significant advantage of RAID, particularly in data-intensive tasks requiring high-speed read and write operations. RAID achieves this through a process called striping, where data is split into smaller blocks and distributed across multiple disks, allowing parallel access to the data.

RAID 0, known for its striping technique without redundancy, provides the highest performance boost. Since data is distributed across multiple disks, multiple drives can be read from or written to simultaneously, significantly improving I/O (Input/Output) speeds. This makes RAID ideal for applications where speed is critical, such as:

Video editing and rendering

- Large databases with frequent access requests
- Scientific simulations requiring rapid data access
- Even RAID levels that focus on redundancy, such as RAID 10 (Mirroring and Striping), maintain high performance while providing fault tolerance, balancing speed and reliability effectively.

3. Scalability

Scalability refers to the ability of a RAID system to expand and accommodate larger storage needs as data requirements grow. RAID offers the flexibility to add more disks to an existing RAID array, increasing both storage capacity and performance without the need for significant system reconfiguration. For example, RAID 5 and RAID 6 are commonly used in scalable environments where data storage needs can grow over time. By adding additional drives, the RAID array can increase its storage capacity while maintaining redundancy and data protection.

This scalability makes RAID particularly beneficial for businesses and organizations that deal with large volumes of data, such as:

- Cloud storage providers
- Media and entertainment industries
- Scientific research organizations with expanding datasets
- Scalable RAID systems help organizations plan long-term data strategies, ensuring they can expand their storage infrastructure without replacing existing systems.

4. Data Protection

Data protection is a core benefit of RAID, ensuring critical data remains safe from hardware failures and corruption. Data protection in RAID is achieved through redundancy mechanisms, such as mirroring or parity storage, which safeguard data integrity.

RAID 1 offers maximum data protection by creating an identical copy of all data on a secondary disk. Meanwhile, RAID 5 and RAID 6 use parity data spread across multiple disks to protect against data loss even when one or two drives fail, respectively.

The importance of data protection through RAID is evident in sectors where data integrity is crucial, such as:

- Financial institutions where transaction data must be preserved accurately.
- Healthcare where patient records need to be secure and recoverable.
- Government agencies managing sensitive data and classified information.
- By ensuring data remains intact and recoverable after hardware malfunctions, RAID plays a crucial role in data governance, compliance, and overall system security.

RAID offers a comprehensive solution for modern data storage challenges by providing fault tolerance, improved performance, scalability, and data protection. These benefits make RAID indispensable for both personal computing and large-scale enterprise systems where data reliability, speed, and capacity are essential. By leveraging RAID technology, organizations can ensure continuous operation, safeguard critical data, and handle expanding storage needs effectively, contributing to greater efficiency and system resilience in modern IT environments.

Limitations and Risks of RAID

While RAID provides significant advantages, it also has certain limitations:

1. Hardware Costs: RAID setups often require multiple disks, increasing initial costs.
2. Complexity: Higher RAID levels like RAID 5 and RAID 6 require more complex setup and management.
3. Data Recovery Challenges: If multiple disks fail in lower RAID levels, data recovery can be difficult.
4. Performance Impact: Some RAID configurations, such as RAID 5 and RAID 6, can experience reduced write speeds due to parity calculations.

RAID in Modern Operating Systems

RAID (Redundant Array of Independent Disks) plays a vital role in modern operating systems by providing data redundancy, performance enhancement, and fault tolerance. Modern operating systems support both software-based RAID and hardware-based RAID solutions, each suited for different use cases and performance needs. This essay explores the role of RAID in contemporary operating systems, focusing on software RAID tools available across major platforms and comparing them with hardware RAID solutions commonly used in enterprise environments.

Software-Based RAID in Modern Operating Systems

Software RAID is a RAID implementation managed by the operating system without requiring dedicated hardware components. It uses system resources (CPU and RAM) to control the RAID configurations, making it a cost-effective choice for personal use, small businesses, and non-critical data protection tasks. The following are the features of Software RAID:

- No additional hardware components required.
- Easier setup through OS interfaces.
- More flexible for personal and small-scale setups.

Modern operating systems have built-in tools to manage RAID configurations, each providing unique functionalities for different user needs.

1. Windows Storage Spaces

Windows Storage Spaces is a built-in feature in Windows operating systems designed to provide RAID-like capabilities for data redundancy and storage optimization. Users can create pools of physical drives and configure them for various RAID levels, such as:

- Two-way and Three-way Mirroring (RAID 1 equivalent): Duplicates data across multiple drives for redundancy.
- Parity (RAID 5 equivalent): Spreads data and parity information across multiple disks.
- Simple (RAID 0 equivalent): Provides striping without redundancy for performance improvements.

Advantages:

- Integrated into Windows.
- User-friendly with a graphical interface.
- Suitable for non-technical users requiring basic RAID setups.

Limitations:

- Performance dependent on system resources.
- Limited advanced RAID configurations compared to hardware RAID.

2. Linux mdadm (Multiple Device Administration)

mdadm is a powerful RAID management tool for Linux systems that allows users to create, manage, and monitor software RAID arrays directly from the command line. It supports a wide range of RAID levels, including RAID 0, 1, 5, 6, and 10.

Advantages:

- Flexible and highly configurable for advanced RAID setups.
- Open-source and widely supported in Linux distributions.
- Supports hot-swapping and fault monitoring.

Limitations:

- Command-line interface may be challenging for beginners.
- Relies on system resources, which may reduce performance in heavy workloads.

3. macOS RAID Utility

macOS RAID Utility provides basic RAID management features within macOS. It supports RAID 0 (striping), RAID 1 (mirroring), and RAID 10 (a combination of mirroring and striping).

Advantages:

Built directly into macOS.

- Simple graphical interface for RAID management.
- Good for creative professionals requiring data redundancy.

Limitations:

- Limited to basic RAID configurations.
- Not suitable for complex enterprise-level RAID setups.

Hardware-Based RAID in Enterprise Environments

While software RAID is ideal for personal and small-scale use, hardware RAID controllers are often preferred in enterprise environments due to their superior performance and fault isolation. Hardware RAID involves a dedicated RAID controller card that manages all RAID operations independently from the operating system.

Features of Hardware RAID:

- Independent RAID processing, reducing CPU load.
- Faster performance due to specialized RAID processors.
- Advanced features like battery-backed cache and automatic rebuilds.

Advantages:

- Enhanced reliability and performance.
- Better fault tolerance and data recovery features.
- Designed for critical business environments (e.g., data centers).

Limitations:

- Higher cost due to dedicated hardware.
- Requires specialized RAID cards and maintenance knowledge.

Table 1 Comparison between Software RAID and Hardware RAID

Feature	Software RAID	Hardware RAID
Performance	Relies on system CPU and RAM.	Dedicated RAID controller, faster.
Cost	Free with OS tools.	Expensive due to specialized hardware.
Ease of Use	Easier setup in modern OS interfaces.	Complex setup, requires expertise.
Fault Tolerance	Limited to OS capabilities.	Advanced fault isolation and caching.
Use Case	Personal use, small businesses.	Enterprise servers, critical data centers.

RAID is a foundational technology in data storage that enhances performance, capacity, and fault tolerance by combining multiple disks into a single logical storage unit. Different RAID levels offer varying balances of speed, redundancy, and storage efficiency, making RAID suitable for various applications from personal use to enterprise data centers. Understanding RAID is essential for IT professionals managing data integrity and system reliability in modern computing environments.

File System Implementation

A file system handles the storage, organization, and retrieval of data on storage devices. It provides an abstraction layer for users and applications to manage data efficiently.

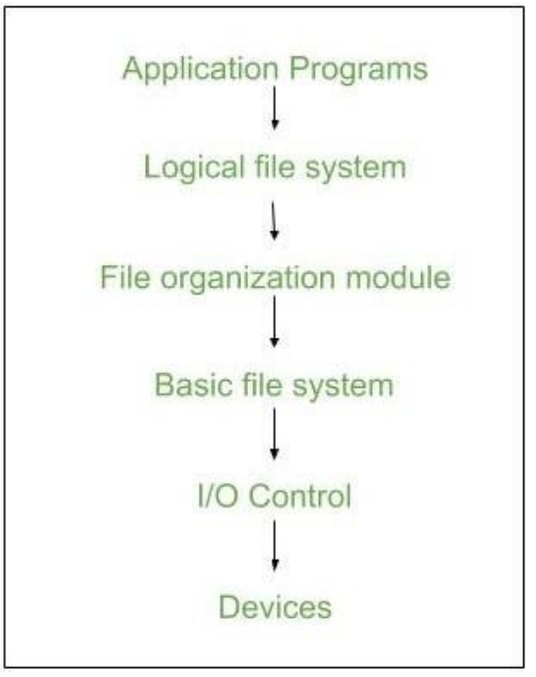


Figure 9 Abstraction Layer for File System

Key Components

File systems are essential components of modern operating systems, responsible for managing how data is stored, accessed, and organized on storage devices. Effective file management ensures data integrity, security, and efficient storage utilization. This essay explores the key components of file systems, common file systems used today, and file system operations crucial for data handling.

A file system is structured with core components that work together to organize and manage data on storage devices effectively. These components include:

1. File Control Block (FCB)

The File Control Block (FCB) is a data structure used by the operating system to store metadata about a file. It acts as a file descriptor and contains critical information, such as:

- File Name: The human-readable identifier for the file.
- File Size: The total storage space occupied by the file.
- Access Permissions: Controls who can read, write, or execute the file.
- File Location: Indicates where the file is stored on the disk.

The FCB is essential for the operating system to manage file access, enforce permissions, and locate data on the storage device.

2. Directory Structure

A directory structure organizes files and directories in a hierarchical format, making it easier for users and the operating system to locate and manage data. There are several types of directory structures:

- Single-Level Directory: A flat structure where all files are stored in a single directory.
- Two-Level Directory: Separates user directories from files.
- Tree-Structured Directory: A hierarchical tree where directories can contain subdirectories and files (most commonly used).
- Acyclic Graph Directory: Allows shared files using links but prevents cyclic references.

This structure enables the creation of organized file storage systems, allowing better accessibility and file grouping based on categories or users.

3. File Allocation Table (FAT)

The File Allocation Table (FAT) is a data structure that maps the physical storage locations of file blocks on a disk. It maintains a table that links each file to the physical blocks it occupies. Key responsibilities include:

- Tracking Free and Occupied Blocks: The FAT keeps track of which disk blocks are available and which are in use.
- Mapping File Data: It provides a chain of pointers for file data blocks.
- Simple File Management: FAT simplifies how the operating system locates and manages files.
- FAT-based systems are widely used for their simplicity but face limitations with modern storage demands.

Common File Systems

File systems differ in design, performance, compatibility, and features. Below are some of the most commonly used file systems in modern operating systems:

1. FAT32 (File Allocation Table 32)

FAT32 is a basic file system introduced in early versions of Windows. It uses a simple table to map files and their data blocks on the disk.

Pros:

- Simple and widely supported across multiple platforms.
- Easy to implement and manage.

Cons:

- Limited to a maximum file size of 4GB and total storage capacity of 2TB.
- Lacks advanced security features like journaling.
- Use Case: USB drives, memory cards, and legacy systems due to its compatibility.

2. NTFS (New Technology File System)

NTFS is the default file system for modern Windows operating systems, designed to support high-performance and secure storage management.

Pros:

- Supports large files and storage volumes (up to 16EB).
- Built-in journaling for improved data integrity.
- Advanced security features, including file permissions and encryption.

Cons:

- More complex and resource-intensive.
- Limited cross-platform support outside Windows.
- Use Case: Windows operating systems, large data servers, and secure storage environments.

3. EXT4 (Fourth Extended File System)

EXT4 is the default file system for most Linux distributions and is known for its reliability and performance.

Pros:

- Journaling for data protection and recovery.
- Supports large files and volumes (up to 16TB).
- Efficient with reduced fragmentation.

Cons:

- Limited compatibility with non-Linux platforms.
- Use Case: Linux-based systems, servers, and open-source environments requiring stable and efficient file storage.

4. HFS+ (Hierarchical File System Plus)

HFS+ is a file system designed by Apple for use in macOS. It supports journaling for enhanced data integrity.

Pros:

- Optimized for Apple devices and workflows.
- Includes journaling for fault tolerance.

Cons:

- Limited support outside of macOS.
- Being replaced by Apple's newer APFS in modern macOS systems.
- Use Case: Apple devices such as MacBooks and iMacs.

File System Operations

File systems provide various operations for managing data, ensuring secure and efficient file handling.

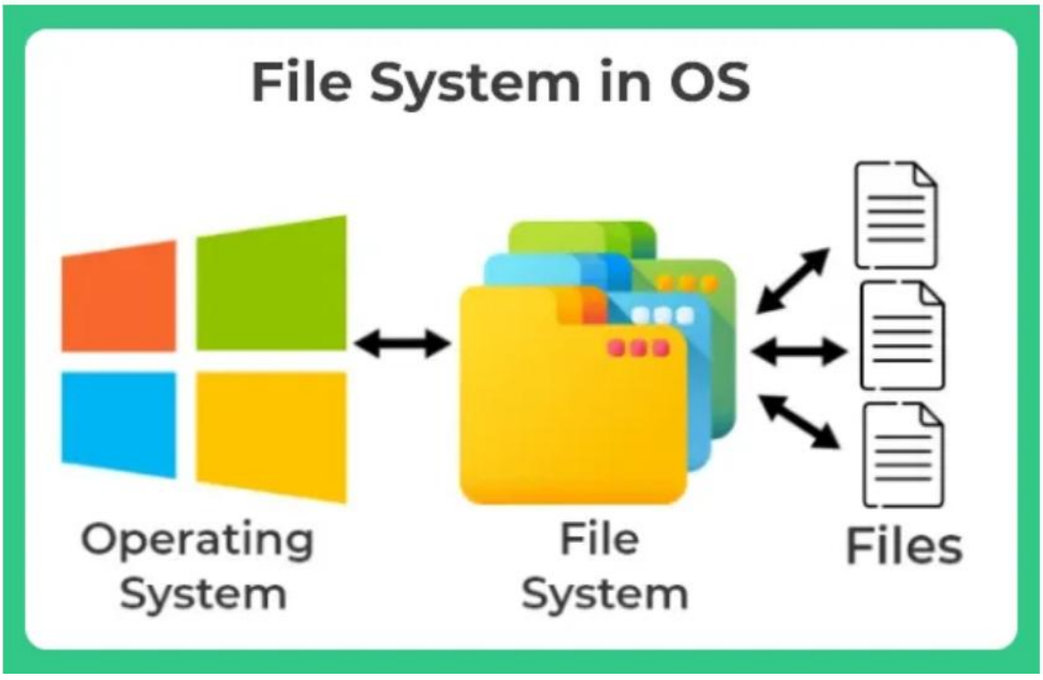


Figure 10 File System in OS

Key file system operations include:

1. File Creation

The file creation process involves generating a new file with a unique identifier. This process includes:

- Allocating an entry in the directory structure.
- Creating a new File Control Block (FCB) with metadata.
- Assigning storage blocks for the file's data.

2. File Reading/Writing

Reading and writing are fundamental operations for accessing and modifying data within a file.

- Read: The system locates the file's physical storage blocks and retrieves the data.
- Write: The system modifies the data blocks and updates the FAT or equivalent structure.

Efficient file reading and writing involve buffering, caching, and pre-fetching techniques to optimize performance.

3. File Deletion

File deletion involves removing a file from the directory and freeing its storage space for reuse.

- The FCB and associated entries in the directory structure are removed.
- Blocks used by the file are marked as free in the FAT.

Deleted files can sometimes be recovered using specialized tools unless securely overwritten.

4. Permission Management

Permission management controls user access rights to files and directories, ensuring data security. Common permission categories include:

- Read (R): Allows viewing the file's content.
- Write (W): Allows modifying the file's content.
- Execute (X): Allows running the file as a program (for executable files).

Permission management is essential for enforcing security policies and preventing unauthorized access to sensitive data.

File systems are fundamental components of modern operating systems, ensuring the organized storage, retrieval, and protection of data. Key components like the File Control Block (FCB), Directory Structure, and File Allocation Table (FAT) work together to manage how data is stored and accessed. Various file systems, including FAT32, NTFS, EXT4, and HFS+, offer unique features suited for different platforms and use cases. By supporting essential file operations such as creation, reading/writing, deletion, and permission management, file systems provide a robust framework for data management in both personal and enterprise computing environments.

Storage Management System

Storage management is a fundamental aspect of modern operating systems, responsible for organizing, storing, and retrieving data efficiently on various storage devices. Proper storage management ensures data integrity, optimal performance, and high availability, making it crucial for both personal computing and enterprise environments. This essay explores the key components of storage management, including disk scheduling algorithms, RAID (Redundant Array of Independent Disks), and file systems, emphasizing their importance in modern operating systems.

1. Disk Scheduling Algorithms

Disk scheduling algorithms are essential for managing how data is accessed on storage devices, particularly hard disk drives (HDDs). Since HDDs rely on mechanical disk heads to read and write data, minimizing the movement of these heads can significantly enhance performance and reduce access time. Key disk scheduling algorithms include:

1. **First-Come, First-Served (FCFS):**

- Serves requests in the order they arrive.
- Simple but inefficient when handling large queues due to high seek time.
- Shortest Seek Time First (SSTF):
- Selects the request closest to the current disk head position.
- Reduces seek time but may lead to starvation for distant requests.

2. **SCAN (Elevator Algorithm):**

- Moves the disk head back and forth like an elevator, servicing requests in both directions.

Balances efficiency but can still favor middle tracks.

3. **C-SCAN (Circular SCAN):**

- Similar to SCAN but moves in one direction only and returns to the beginning without servicing.

Provides uniform wait times across all tracks.

4. **LOOK and C-LOOK:**

- Variations of SCAN and C-SCAN that stop once the last request in a direction is serviced.

These algorithms aim to reduce seek time, enhance I/O throughput, and ensure efficient disk utilization, especially in environments handling large volumes of data.

2. RAID (Redundant Array of Independent Disks)

RAID is a data storage virtualization technology that improves performance, redundancy, and fault tolerance by combining multiple physical disks into a single logical unit. RAID configurations balance speed and data protection, making them essential for modern storage systems. Key RAID levels include:

1) **RAID 0 (Striping):**

- Data is divided across multiple disks for improved performance.
- No redundancy, so data loss occurs if one disk fails.

2) **RAID 1 (Mirroring):**

- Data is duplicated across two disks, ensuring fault tolerance.
- Slower write performance and reduced storage capacity since data is duplicated.

3) **RAID 5 (Striping with Parity):**

- Distributes both data and parity across multiple disks.
- Can recover from the failure of a single disk but requires at least three disks.

4) **RAID 6 (Double Parity):**

- Enhanced version of RAID 5 with double parity, allowing recovery from two disk failures.

5) **RAID 10 (1+0):**

- Combines mirroring and striping for both fault tolerance and performance.
- Requires a minimum of four disks.

RAID provides data redundancy, improved read/write performance, and fault tolerance, making it suitable for critical environments such as data centers, enterprise servers, and high-availability systems.

3. File Systems

A file system is a method used by an operating system to organize, manage, and store data on storage devices. It provides a structured way to store and retrieve files, ensuring data integrity and accessibility. Different file systems vary in structure, performance, and compatibility.

Common file systems include:

A. **FAT32 (File Allocation Table 32):**

- A basic, widely supported file system suitable for USB drives and memory cards.
- Limited to a 4GB file size and lacks advanced features like journaling.

B. NTFS (New Technology File System):

- Default file system for Windows with support for journaling, encryption, and large file sizes.
- Enhanced security features but limited cross-platform compatibility.

EXT4 (Extended File System 4):

- Default file system for Linux, known for reliability and journaling support.
- Not natively supported on Windows and macOS.

HFS+ and APFS (Apple File Systems):

- HFS+ was the standard file system for macOS, later replaced by APFS.
- Designed for Apple devices, with optimized performance and security features like file encryption.

Key Components of a File System:

- File Control Block (FCB): Stores metadata like file size, name, and access permissions.
- Directory Structure: Organizes files hierarchically for easier access.
- File Allocation Table (FAT): Maps file blocks to physical storage locations.
- File systems ensure data organization, efficient storage usage, and secure file access across various platforms and storage devices.

Importance of Effective Storage Management

Effective storage management is crucial in modern operating systems for several reasons:

- Data Integrity: Proper storage management minimizes data corruption risks and ensures files remain intact even during power failures or system crashes (e.g., through journaling in NTFS and EXT4).
- Performance Optimization: Disk scheduling algorithms and RAID configurations help improve data access speed and system responsiveness.
- Fault Tolerance and Redundancy: RAID configurations like RAID 1 and RAID 5 protect against data loss by creating redundant copies of data.
- Efficient Resource Utilization: File systems like NTFS and EXT4 maximize disk space usage and manage fragmentation.
- High Availability: Storage management strategies like RAID and journaling ensure data remains accessible even in case of hardware failure.

5.3 Analysis of File Systems in Modern Operating Systems

A file system is a fundamental component of an operating system (OS) that manages how data is stored, organized, accessed, and secured on storage devices such as hard drives, SSDs, and external storage media. Modern operating systems use file systems to ensure efficient data management, integrity, and security.

1. File System

A file system provides a structured method for storing and managing data on a storage device. It defines how data is stored using logical structures like directories and files and maps them to physical disk blocks.

Core Functions of a File System:

- Data Organization: Manages files and directories logically.
- Data Storage: Allocates disk space effectively.
- Data Retrieval: Enables efficient access to stored data.
- Security: Enforces file permissions and access controls.
- Error Handling: Detects and resolves data corruption issues.

2. Components of File Systems

- File Control Block (FCB): Stores metadata such as file name, size, and permissions.
- Directory Structure: Organizes files hierarchically for easier navigation.
- File Allocation Table (FAT): Maps file data blocks on disk.
- Index Node (Inode): Used in UNIX/Linux for file metadata and disk block references.
- Journal: Logs file operations to prevent data corruption during crashes.

3. Types of File Systems in Modern Operating Systems

- Modern operating systems support multiple file systems suited for specific purposes:
- FAT32: (Windows, external drives) – Simple but limited to 4GB file size.
- NTFS: (Windows) – Journaling, encryption, and large file support.
- EXT4: (Linux) – Journaling, large file support, backward compatibility.
- HFS+: (macOS) – Journaling and Apple device optimization.
- APFS: (macOS/iOS) – Modern encryption, snapshots, and SSD optimization.
- XFS: (Linux) – High-performance, large file support.
- Btrfs: (Linux) – Advanced features like compression, snapshots, and error correction.

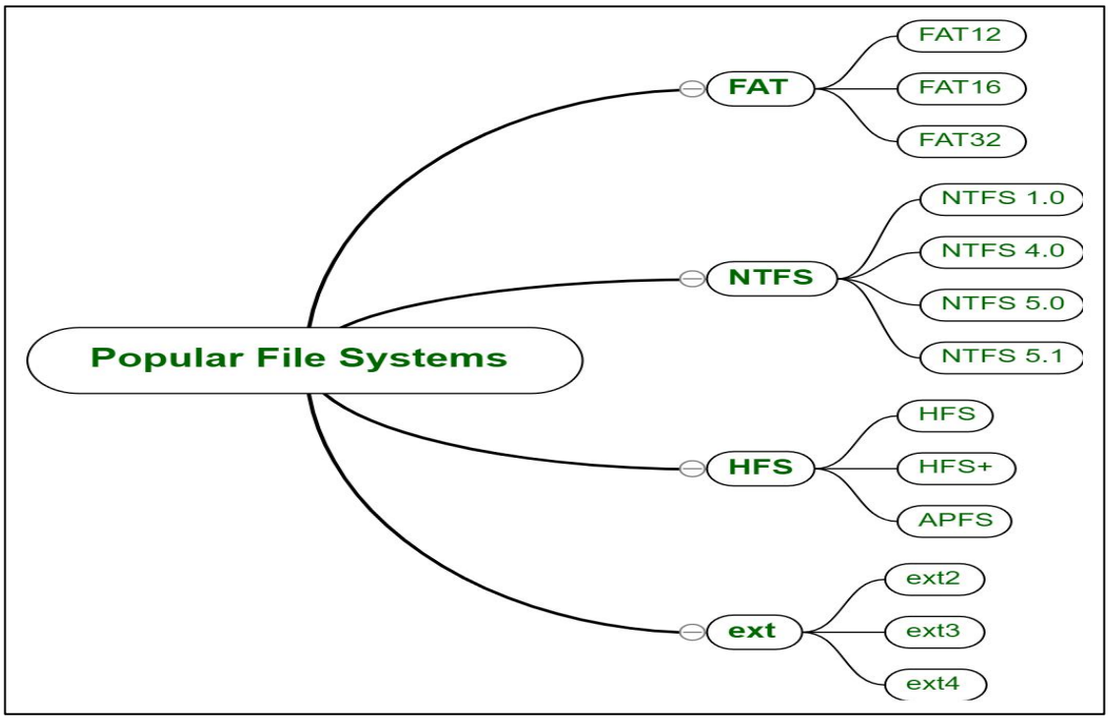


Figure 11 Popular File System

4. File Allocation Methods in Modern File Systems

File allocation methods determine how disk space is allocated to files:

- Contiguous Allocation: Files stored in continuous memory blocks. (Fast but prone to fragmentation)
- Linked Allocation: Files stored in scattered blocks linked by pointers. (No fragmentation but slower access)
- Indexed Allocation: Files stored with an index table for block references. (Direct access but limited by index size)

5. Features of Modern File Systems

- Journaling: Logs file operations to prevent corruption. (e.g., NTFS, EXT4, HFS+)
- Encryption: Provides data protection. (e.g., NTFS BitLocker, APFS)
- Compression: Reduces file sizes. (e.g., NTFS, Btrfs)
- Data Integrity and Error Detection: Detects and corrects data corruption. (e.g., Btrfs, ZFS)
- Snapshots and Versioning: Captures point-in-time data copies. (e.g., APFS, Btrfs)

6. Performance Factors in File Systems

- Speed and Efficiency: Influenced by file structure and journaling overhead.
- Scalability: NTFS and EXT4 support large file sizes, while FAT32 is limited.
- Fragmentation: NTFS includes built-in defragmentation tools.
- Reliability: Journaling and RAID integration improve fault tolerance.

7. File System Security Considerations

Modern file systems implement security features such as:

- Permissions and Access Control: NTFS supports detailed ACLs.
- Encryption: APFS and NTFS offer full disk encryption.
- Data Redundancy: RAID ensures fault tolerance through disk mirroring and striping.

8. Comparative Analysis of Modern File Systems

File System	Platform	Max File Size	Security	Journaling	Compression
FAT32	Windows/Linux	4GB	No	No	No
NTFS	Windows	16EB	Yes	Yes	Yes
EXT4	Linux	16TB	Limited	Yes	No
HFS+	macOS	8EB	Yes	Yes	No
APFS	macOS/iOS	8EB	Yes	Yes	No
Btrfs	Linux	16EB	Yes	Yes	Yes

Figure 12 Comparison table of modern file systems

Analyzing modern file systems reveals the importance of balancing performance, security, and scalability. Systems like NTFS, EXT4, and APFS offer advanced security and performance features, while FAT32 remains a simple choice for cross-platform compatibility. Selecting the right file system depends on platform requirements, data management needs, and performance expectations.

EXERCISE

Answer all questions.

1. True/False: Disk scheduling algorithms are used to minimize disk head movement and improve I/O performance.
2. True/False: RAID 0 provides both data redundancy and fault tolerance.
3. True/False: NTFS is commonly used in Windows systems and supports journaling for better data integrity.
4. True/False: RAID 1 involves data mirroring where data is copied to two or more disks for fault tolerance.
5. True/False: FAT32 supports unlimited file sizes and is the most advanced file system available.
6. True/False: Disk scheduling algorithms like SSTF always prevent starvation issues in disk access.
7. True/False: RAID 5 requires a minimum of three disks and can tolerate a single disk failure.
8. True/False: A File Control Block (FCB) stores metadata such as file size, name, and access permissions.
9. True/False: The EXT4 file system is the default for Linux systems and supports journaling for reliability.
10. True/False: HFS+ is commonly used in Windows systems due to its high compatibility.
11. True/False: The primary role of a directory structure in a file system is to organize and manage file locations.
12. True/False: RAID 6 can only tolerate a single disk failure.
13. True/False: The File Allocation Table (FAT) maps physical storage locations for files on the disk.
14. True/False: Journaling in file systems helps track file changes to prevent data corruption during crashes.
15. True/False: Software RAID relies on a dedicated hardware controller for RAID management.

5.4 SUMMARY

File management is a critical function of modern operating systems, ensuring efficient organization, storage, retrieval, and protection of data stored on disk drives and other storage media. The primary components of file management include the file system structure, storage management techniques, and an analysis of file systems used in contemporary operating systems. Effective file management ensures data integrity, optimized performance, and secure data access across different platforms.

1. File System Structure: Directory Structures and File Allocation Methods

The file system structure provides a framework for storing and managing data on a storage device. It determines how files are named, organized, and accessed. Two key elements of file system structure include directory structures and file allocation methods.

Directory Structures:

A directory structure organizes files in a logical, hierarchical manner, making it easier for users and the operating system to locate and manage files. Common types include:

- Single-Level Directory: All files are stored in a single directory, limiting file organization.
- Two-Level Directory: Separates user directories from files, providing basic isolation between users.
- Hierarchical (Tree) Directory: The most commonly used structure, where directories can contain subdirectories and files, forming a tree structure.
- Acyclic Graph: Allows shared files through links but prevents cyclic references.
- A well-organized directory structure improves data retrieval efficiency and simplifies system navigation.

File Allocation Methods:

- File allocation methods determine how files are stored on the physical disk. These methods include:
- Contiguous Allocation: Files are stored in continuous disk blocks. Fast access but prone to fragmentation.
- Linked Allocation: Each file block points to the next, avoiding fragmentation but with slower random access.
- Indexed Allocation: An index block stores pointers to all data blocks of a file, balancing speed

and flexibility.

Efficient allocation methods help minimize fragmentation and optimize disk space usage.

2. Storage Management: Disk Scheduling, RAID, and File System Implementation

Storage management involves the optimization of disk space, data access speed, and fault tolerance to ensure efficient data storage and retrieval. It includes disk scheduling algorithms, RAID (Redundant Array of Independent Disks), and file system implementation strategies.

Disk Scheduling Algorithms:

Disk scheduling algorithms manage how data is accessed from storage devices, especially hard disk drives (HDDs) where mechanical movement affects speed.

Key algorithms include:

- FCFS (First-Come, First-Served): Processes requests in the order they arrive but inefficient with large queues.
- SSTF (Shortest Seek Time First): Prioritizes the nearest request but may cause starvation.
- SCAN (Elevator Algorithm): Moves the disk head back and forth, servicing requests along the way.
- C-SCAN (Circular SCAN): Services requests in a circular motion, improving fairness.
- These algorithms aim to reduce disk head movement, minimizing seek time and enhancing input/output (I/O) performance.

RAID (Redundant Array of Independent Disks):

RAID is a data storage technology that enhances performance, redundancy, and fault tolerance by combining multiple disks into a single logical unit. Common RAID levels include:

1. RAID 0 (Striping): Improves performance by splitting data across multiple disks but lacks redundancy.
2. RAID 1 (Mirroring): Duplicates data across disks, ensuring fault tolerance but with reduced storage capacity.
3. RAID 5 (Striping with Parity): Balances performance and redundancy by using parity data.
4. RAID 6 (Double Parity): Enhanced redundancy for critical data with the ability to withstand multiple disk failures.
5. RAID 10: Combines RAID 1 and RAID 0 for both performance and fault tolerance.
6. RAID ensures data redundancy, preventing data loss in case of hardware failure while also improving read/write speeds.

File System Implementation:

File system implementation involves how the operating system manages file storage and retrieval. Core components include:

- File Control Block (FCB): Stores metadata about files, including size, location, and permissions.
- File Allocation Table (FAT): Maps files to physical storage locations.
- Inodes: Used in UNIX-based systems to store file attributes and data block pointers.

File systems such as NTFS (New Technology File System), EXT4 (Extended File System 4), and HFS+ (Hierarchical File System Plus) implement these structures differently, balancing speed, security, and compatibility.

3. Analysis of File Systems in Modern Operating Systems

Modern operating systems use various file systems designed for different use cases, performance needs, and platform compatibility. A comparative analysis reveals the strengths and limitations of popular file systems:

1. FAT32 (File Allocation Table 32):

Simple, widely compatible but limited to a 4GB file size.

2. NTFS (New Technology File System):

Used in Windows, supports journaling, encryption, and large file handling.

3. EXT4 (Extended File System 4):

Default file system for Linux, offering reliability, journaling, and support for large files.

4. HFS+ and APFS (Apple File Systems):

HFS+ supports Apple devices with journaling, while APFS (Apple File System) provides faster performance and better encryption for modern macOS devices.

Key Factors for Evaluating File Systems:

Performance: Speed of file access and data transfer.

Security: Encryption, permissions, and journaling support.

Scalability: Ability to handle large files and storage volumes.

Compatibility: Cross-platform support and ease of data transfer.

Modern file systems are designed to ensure data reliability, fault tolerance, and high-speed access, especially in enterprise environments where data integrity is critical.

Conclusion

File management in modern operating systems involves a combination of file system structures, storage management techniques, and the analysis of various file systems. Efficient directory structures and file allocation methods help organize data logically, while disk scheduling algorithms and RAID configurations optimize storage performance and reliability. Modern file systems like NTFS, EXT4, and APFS provide unique features tailored to different operating environments, ensuring secure, efficient, and scalable data storage solutions. A well-designed file management system is crucial for both everyday computing and enterprise data operations, balancing performance, security, and fault tolerance.



Here are some useful YouTube videos File Management

1) Explaining File System Watch here
2) File System and Threads Watch here
3) File Attributes System. Watch here



Review Question

Exercise 1: Matching Pairs

Instruction: Match the terms with their correct descriptions

FAT32		Default file system for macOS with journaling support.
NTFS		Used in Linux for large files and efficient journaling.
EXT4		Basic file system with limited file size (4GB).
HFS+		Advanced file system for Windows with encryption support.

Exercise 2: True/False Quiz (Storage Management Concepts)

1) True/False: RAID 0 provides fault tolerance.
2) True/False: NTFS supports file encryption and journaling.
3) True/False: Disk scheduling algorithms help reduce disk head movement.
4) True/False: The FAT32 file system can handle files larger than 4GB.
5) True/False: EXT4 is the default file system in Windows

Exercise 3: Fill in the Blanks (Concept Recall)

1) The _____ file system is used in Windows and supports journaling and encryption. (Answer: NTFS)
2) _____ is a RAID level that mirrors data for fault tolerance.
3) Disk scheduling algorithms help optimize _____ performance.
4) _____ allocation stores files in continuous memory blocks.
5) _____ is a Linux file system known for reliability and journaling.